

IEEE 830 / ISO 29148 STANDARD

Software Requirements Specification (SRS)

Formal Functional & Non-Functional Specifications for VarsityConnect

PROJECT

VarsityConnect

DISCIPLINE

Senior Software Eng.

COMPLIANCE

IEEE 830 / ISO 29148

STATUS

Approved &
Implemented

Project: VarsityConnect Campus Community Platform

Course / Discipline: Senior Software Engineering (SSE)

Standard Compliance: IEEE 830 / ISO/IEC/IEEE 29148

System Version: 1.0.0

Status: Approved & Implemented

1. Introduction

1.1 Purpose

The purpose of this document is to specify the complete software and system requirements for **VarsityConnect**, an integrated web-based campus community platform. This Software Requirements Specification (SRS) serves as the definitive reference for developers, software architects, quality assurance engineers, university stakeholders, and academic evaluators. It defines the functional capabilities, non-functional performance and security guarantees, data schema, system interfaces, and verification criteria.

1.2 Project Scope

VarsityConnect addresses the fragmented digital experience of university students by consolidating four critical student workflows into a single authenticated, mobile-responsive portal:

- Academic Resource Repository:** Centralized sharing, indexing, previewing, and downloading of course notes, textbooks, and past examination papers.
- Student Peer-to-Peer Marketplace:** A campus-localized exchange for textbooks, electronics, dorm essentials, and academic services with real-time stock and condition tracking.
- Campus Opportunities & Housing Directory:** Direct access to student employment, internships, part-time jobs, and off-campus student accommodation listings.
- Direct Intra-Campus Messaging:** A secure, one-to-one communication hub connecting buyers, sellers,

study partners, and job seekers without disclosing personal phone numbers or external social media handles. 5. **Centralized Student Dashboard & Profile:** High-level metrics tracking user listings, uploads, job postings, and quick actions.

1.3 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
SRS	Software Requirements Specification
P2P	Peer-to-Peer (student-to-student transactions and communications)
ORM	Object-Relational Mapping (Django ORM)
CRUD	Create, Read, Update, Delete data operations
RBAC	Role-Based Access Control
CSRF	Cross-Site Request Forgery
XSS	Cross-Site Scripting
NFR	Non-Functional Requirement
RTM	Requirements Traceability Matrix
UI / UX	User Interface / User Experience

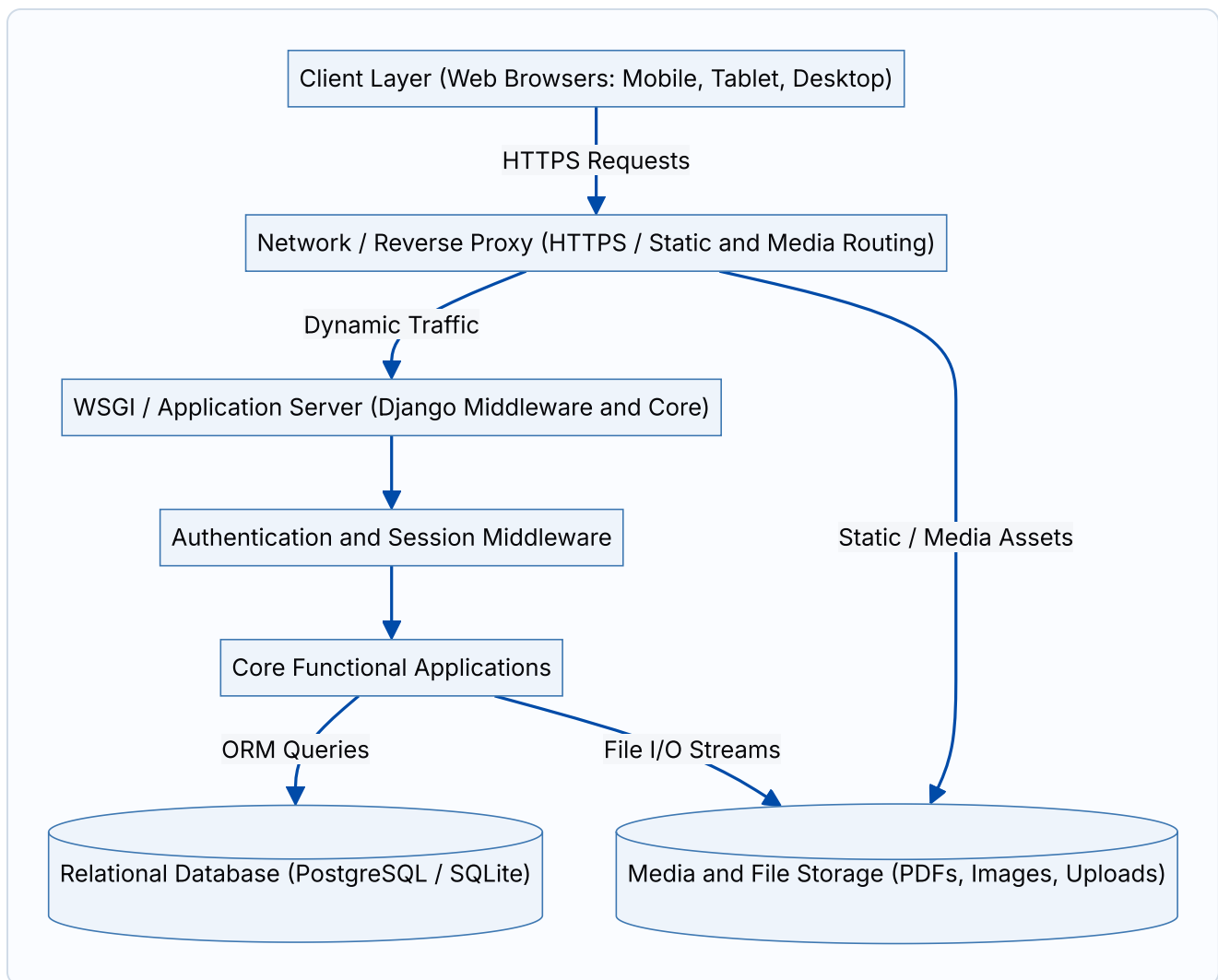
1.4 References

- IEEE Std 830-1998: *IEEE Recommended Practice for Software Requirements Specifications*.
- ISO/IEC/IEEE 29148:2018: *Systems and software engineering — Life cycle processes — Requirements engineering*.
- Django Framework 6.x Software Architecture & Security Documentation.
- Web Content Accessibility Guidelines (WCAG) 2.1 Level AA.

2. Overall Description

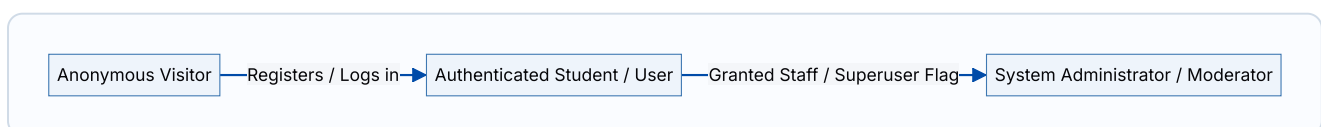
2.1 Product Perspective

VarsityConnect is a multi-tier, server-side rendered web application built using Python/Django, backed by an ACID-compliant relational database (SQLite in development, PostgreSQL in production), and presented through standard-compliant HTML5, CSS3, and JavaScript.



2.2 User Classes and Personas

The system recognizes three primary user classes with distinct privilege levels:



1. Anonymous Visitor (**Visitor**):

2. Unauthenticated student or prospective user.
3. Restricted to landing page, user registration, and authentication login screens.
4. Enforced redirection (`LOGIN_URL = 'login'`) when attempting to access internal endpoints.

5. Authenticated Student (**Student**):

6. Verified student who has completed registration and logged in.
7. Authorized to manage personal profile, upload and download course materials, list items on the marketplace, update listing inventory, apply for or post jobs, and participate in direct messaging.
8. Restricted from editing or deleting resources/listings/jobs created by other students.

9. System Administrator / Moderator (**Staff / Superuser**):

10. High-privilege university or platform administrator.
11. Authorized to access the Django Administration Console (`/admin/`), inspect system logs, moderate abusive content, manage user access, and maintain database integrity.

2.3 Operating Environment

- **Server Operating Systems:** Linux (Ubuntu 22.04 LTS+, Debian 12+), Windows Server, or containerized Docker environments.
- **Runtime Environment:** Python 3.10 to 3.14.
- **Web Application Server:** Gunicorn, Uvicorn, or Django WSGI server.
- **Client Platforms:** Modern evergreen web browsers (Google Chrome 110+, Mozilla Firefox 110+, Apple Safari 16+, Microsoft Edge 110+) across desktop, tablet, and mobile viewports.

2.4 Design and Implementation Constraints

1. **Design System & Visual Consistency:**
2. Must strictly adhere to the university-approved 5-color architectural palette:
 - **Steel Blue:** `#3e78b2` (accents, interactive links, secondary highlights)
 - **Cobalt Royal Blue:** `#004ba8` (primary CTA buttons, active states, key emphasis)
 - **Slate Gray:** `#4a525a` (subtitles, secondary text, metadata labels)
 - **Deep Charcoal:** `#24272b` (cards, structured container borders, dark mode accents)
 - **Midnight Obsidian:** `#07070a` (sidebar shell, primary headers, modal bases)
3. Fluid typography scaling using `clamp()` expressions to guarantee legibility across screen sizes without clipping or artificial overflow.
4. **File Size and Type Limitations:**
5. Academic resource uploads are restricted to `.pdf` format.
6. Image uploads are restricted to raster image formats (`.jpg` , `.jpeg` , `.png` , `.gif` , `.webp`).
7. **Session Management:**
8. Session cookies configured to terminate upon browser window closure (`SESSION_EXPIRE_AT_BROWSER_CLOSE = True`).
9. Secure HTTP-only and SameSite cookie policies for CSRF and session tokens.

2.5 Assumptions and Dependencies

- Users possess standard network connectivity and modern web browsers capable of rendering HTML5, CSS Grid, Flexbox, and executing basic DOM JavaScript.
 - File storage volume has sufficient disk allocation and appropriate read/write POSIX permissions for uploaded student media and PDF documents.
-

3. Specific Functional Requirements

3.1 Module 1: User Authentication & Profile Management (`registration` , `dashboard`)

FR-AUTH-01: User Registration

- **Description:** Prospective users shall be able to register a new platform account by supplying a unique username, valid credentials, and password confirmation.
- **Inputs:** Username, password, password confirmation.
- **Outputs:** Created user record in `auth_user` , flash confirmation message, automatic session initialization, redirection to dashboard.
- **Processing Logic:**
 - Validate username uniqueness and character compliance.
 - Validate password complexity using configured Django validators (`MinimumLengthValidator` , `CommonPasswordValidator` , `NumericPasswordValidator` , `UserAttributeSimilarityValidator`).
 - Hash password using PBKDF2 with SHA-256.
 - Persist user entity and establish an active session.

FR-AUTH-02: User Authentication & Login

- **Description:** Registered users shall authenticate using valid credentials to access protected system features.
- **Inputs:** Username, password, optional `next` URL query parameter.
- **Outputs:** Authenticated session cookie or error feedback ("Invalid username or password.").
- **Processing Logic:** Authenticate credentials against `auth_user` . Sanitize and validate `next` redirection target using `url_has_allowed_host_and_scheme` to prevent open-redirect vulnerabilities.

FR-AUTH-03: Session Termination & Logout Confirmation

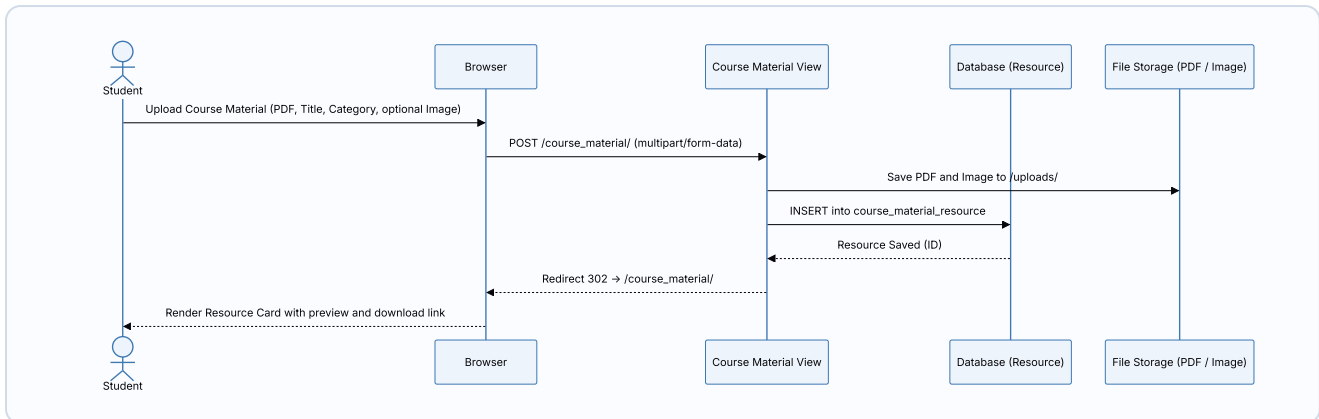
- **Description:** Authenticated users shall be able to securely terminate their session at any time.
- **Inputs:** HTTP POST request to logout endpoint.
- **Outputs:** Flushed session storage, expired session cookie, dedicated logout acknowledgement view (`registration/logout_thanks.html`).
- **Processing Logic:** Enforce HTTP POST method requirement via `@require_POST` to prevent CSRF-based malicious logouts.

FR-AUTH-04: Student Profile Viewer & Aggregated Analytics

- **Description:** The system shall provide a dedicated Profile view displaying the user's identity, join date, and dynamic activity metrics across all modules.
- **Outputs:**
 - Dynamic display name resolution (full name, first name, or fallback username).
 - Truncation protection allowing multi-line wrapping for long names (`overflow-wrap: break-word` , 2-line clamp).

- Exact count of active marketplace listings authored by the user
(`MarketplaceListing.objects.filter(seller=user).count()`).
- Exact count of active job postings authored by the user
(`Job.objects.filter(posted_by=user).count()`).
- Exact count of academic resources shared by the user
(`Resource.objects.filter(uploaded_by=user).count()`).

3.2 Module 2: Academic Course Material Repository (`course_material`)



FR-ACAD-01: Material Listing & Categorization

- **Description:** The system shall display all shared academic study resources in a responsive grid layout.
- **Attributes:** Title, category badge (`TEXTBOOKS` , `NOTES` , `PAST_PAPERS` , `OTHER`), uploader username, publication date, download button.
- **Fallback Media:** If an image is not attached or fails to render, the card must dynamically render an SVG document placeholder rather than an empty void.

FR-ACAD-02: Resource Contribution & Upload

- **Description:** Authenticated students shall be able to upload academic notes, textbook references, and past papers.
- **Inputs:** Resource Title (`max_length=200`), Category choice, PDF document file (`uploads/pdfs/`), optional cover image (`uploads/course_material/`).
- **Processing Logic:** Bind `request.user` to `uploaded_by` . Validate file types. Save file records to disk storage.

FR-ACAD-03: Resource Download Delivery

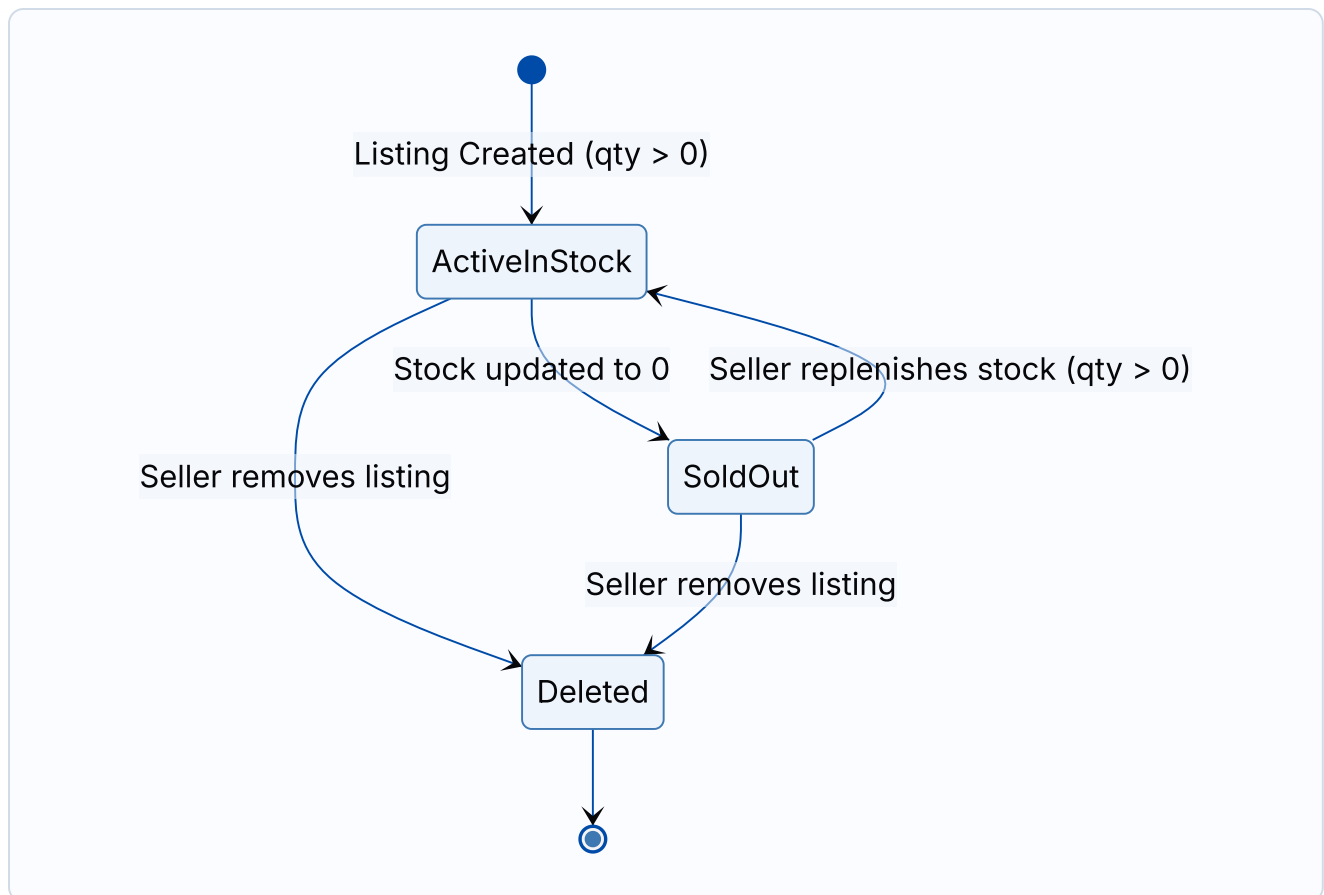
- **Description:** Users shall be able to securely stream or download the PDF document associated with any resource.
- **Inputs:** Resource Primary Key (`pk`).
- **Processing Logic:**
- Verify resource existence via `get_object_or_404` .

- Stream binary payload as an attachment using `FileResponse` with `content_type='application/pdf'`.
- Extract sanitized base filename to prevent path traversal.
- Trigger client-side download toast confirmation.

FR-ACAD-04: Author Content Deletion & File Cleanup

- **Description:** The original uploader of an academic resource shall have permission to permanently delete their uploaded resource.
- **Precondition:** Requesting user must match `resource.uploaded_by`.
- **Processing Logic:** Require HTTP POST. Delete physical PDF file and image file from the file storage before removing the database record to prevent orphan file accumulation.

3.3 Module 3: Student Marketplace & Inventory Management (`marketplace`)



FR-MKT-01: Marketplace Catalog Display

- **Description:** The system shall provide a visual catalog of all active peer-to-peer listings sorted chronologically in descending order.
- **Attributes:** Item title, price (ZAR, formatted as `R<value>`), price suffix (e.g. `/hr`), category badge, stock status pill (`In Stock` / `Sold Out`), seller handle, physical campus location.

- **Visual Assurance:** Support responsive image cards with hover magnification (`scale(1.03)`) and automatic dark obsidian fallback placeholders with category icons.

FR-MKT-02: Listing Creation

- **Description:** Students shall be able to publish items for sale or trade.
- **Inputs:** Title, Category (`textbooks` , `electronics` , `furniture` , `services`), Price, Price Suffix, Quantity Available, Condition, Description, Campus Location, optional Cover Image.
- **Processing Logic:** Bind `request.user` as `seller` . Initialize default seller rating (5.00) and completed sales count (0).

FR-MKT-03: Listing Detail & Inquiry

- **Description:** Prospective buyers shall be able to view full details of any listed item, including comprehensive description, condition, seller stats, and a direct "Contact Seller" CTA button.
- **Processing Logic:** Clicking "Contact Seller" resolves the seller's username and initiates or opens an existing one-to-one conversation via the messaging module (`start_conversation_view`).

FR-MKT-04: Dynamic Stock Management

- **Description:** The author of a listing shall be able to modify the available stock quantity directly from the listing detail page.
- **Inputs:** Positive integer `quantity_available` .
- **Outputs:** Immediate update of the stock badge from `In Stock` to `Currently sold out` if quantity reaches zero.

FR-MKT-05: Listing Deletion

- **Description:** Sellers shall be able to remove their own listings from the marketplace.
- **Precondition:** Requesting user must match `listing.seller` . Associated uploaded images must be deleted from storage.

3.4 Module 4: Campus Jobs & Opportunities Directory (`housing`)

FR-JOB-01: Job & Internship Board

- **Description:** The system shall display available student job openings, internships, peer tutoring roles, and campus opportunities.
- **Attributes:** Job title, organization/employer, physical/campus location, compensation/pay rate, publication timestamp.

FR-JOB-02: Job Posting Creation

- **Description:** Students or verified campus representatives shall be able to submit job postings.
- **Inputs:** Title, Organization, Location, Pay structure, Detailed Description.
- **Processing Logic:** Associate authenticated user as `posted_by` . Order postings chronologically.

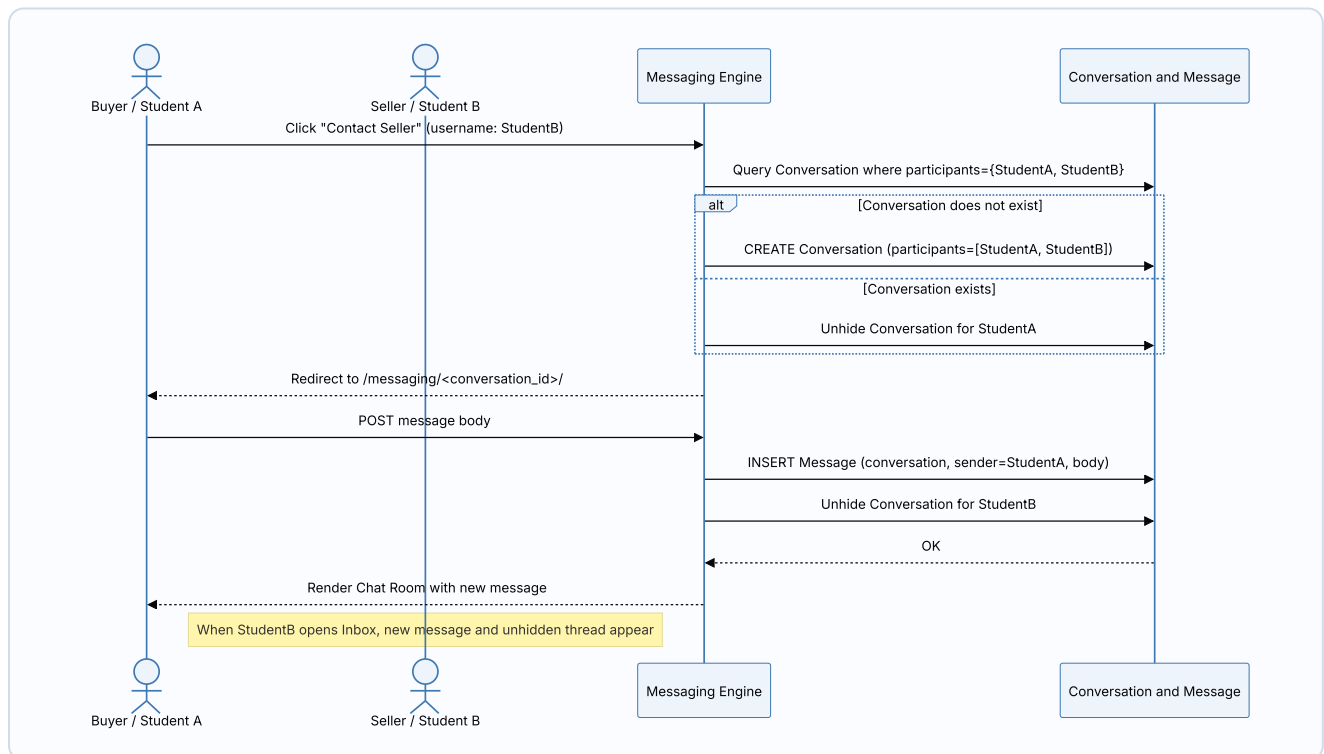
FR-JOB-03: Job Details & Application Routing

- **Description:** Users shall be able to open a dedicated job detail view to inspect comprehensive job requirements and terms.

FR-JOB-04: Job Deletion

- **Description:** Posters shall be able to delete obsolete or filled job postings they authored.

3.5 Module 5: Direct Intra-Campus Messaging System (`messaging`)



FR-MSG-01: Conversation Initialization

- **Description:** The system shall allow users to initiate a private conversation with any other user by username.
- **Precondition:** Users cannot initiate a conversation with themselves (`recipient ≠ request.user`).
- **Processing Logic:** Check for an existing conversation entity containing both participants. If none exists, instantiate a new `Conversation` and add both users to the `participants` relation.

FR-MSG-02: Unified Conversation Inbox

- **Description:** Users shall view a unified list of active conversations sorted by most recent message activity (`-updated_at`).
- **Attributes:** Other participant username, user initial avatar, snippet of the latest message, timestamp.
- **Privacy Filtering:** Conversations marked in `hidden_for` for the requesting user shall be excluded from the view.

FR-MSG-03: Real-Time Chat Room

- **Description:** Participants shall be able to view message history and send new messages in a responsive conversational thread.
- **Processing Logic:**
- Enforce participant authorization (`participants.contains(request.user)`).
- Submitting a new message unhides the conversation for the other participant, ensuring replies are never lost.
- Sort messages chronologically by timestamp (`timestamp ASC`).

FR-MSG-04: Non-Destructive Message Deletion

- **Description:** Users shall be able to delete individual messages or entire conversations from their own view without destroying the conversation history for the other participant.
 - **Processing Logic:** Add `request.user` to the `hidden_for` ManyToMany relationship on `Message` or `Conversation` . Filter queries using `.exclude(hidden_for=request.user)` .
-

3.6 Module 6: Dashboard & Activity Summaries (`dashboard`)

FR-DASH-01: Global Platform Metrics

- **Description:** The home dashboard shall present high-level platform activity metrics to orient the student.
- **Metrics:** Total active marketplace listings, total academic resources shared, total active jobs posted.

FR-DASH-02: Direct Navigation & Action Shortcuts

- **Description:** The dashboard shall provide one-click action buttons ("Find Books & Gear", "Browse Study Notes", "Explore Student Jobs") and quick action links ("List an Item", "Upload Notes", "Post a Job").
-

4. Non-Functional Requirements (NFR)

4.1 Security & Data Integrity Requirements

ID	Requirement	Implementation Specification
NFR-SEC-01	Authentication Protection	All functional views (except login, register, and static assets) must be guarded with <code>@login_required</code> decorators. Unauthorized requests are redirected to <code>LOGIN_URL</code> .
NFR-SEC-02	Object-Level Authorization	Resource deletion, listing deletion, and job deletion endpoints must verify ownership (<code>get_object_or_404(Model, pk=pk, author=request.user)</code>) to prevent Insecure Direct Object References (IDOR).
NFR-SEC-03	CSRF Prevention	All POST forms must inject <code>{% csrf_token %}</code> validated by <code>CsrfViewMiddleware</code> . State-mutating endpoints must enforce POST via <code>@require_POST</code> .
NFR-SEC-04	Credential Security	Passwords must never be stored in plaintext. Passwords must be hashed using Django's PBKDF2-SHA256 implementation with individual salts.
NFR-SEC-05	XSS & Injection Defense	All dynamic template variables must be automatically escaped by Django's template engine. Database interactions must strictly use parameterized ORM queries.
NFR-SEC-06	Session Lifetime	Sessions must invalidate when the student closes their web browser (<code>SESSION_EXPIRE_AT_BROWSER_CLOSE = True</code>).

4.2 Performance & Responsiveness Requirements

ID	Requirement	Target Metric
NFR-PERF-01	Page Render Latency	Server response time (TTFB) under standard campus loads (<50 concurrent users) shall not exceed 800ms .
NFR-PERF-02	Static Asset Delivery	CSS and image assets shall be cached by client browsers; layout CSS files shall be minified and bundled inline or preloaded.
NFR-PERF-03	Media Streaming	File downloads for PDF documents up to 25 MB must commence transmission within 1.5 seconds .
NFR-PERF-04	Database Efficiency	Multi-table joins (e.g., participants in conversations) must utilize indexed foreign keys and Django <code>prefetch_related</code> / <code>select_related</code> where appropriate.

4.3 Usability, Accessibility, and Responsive UI Requirements

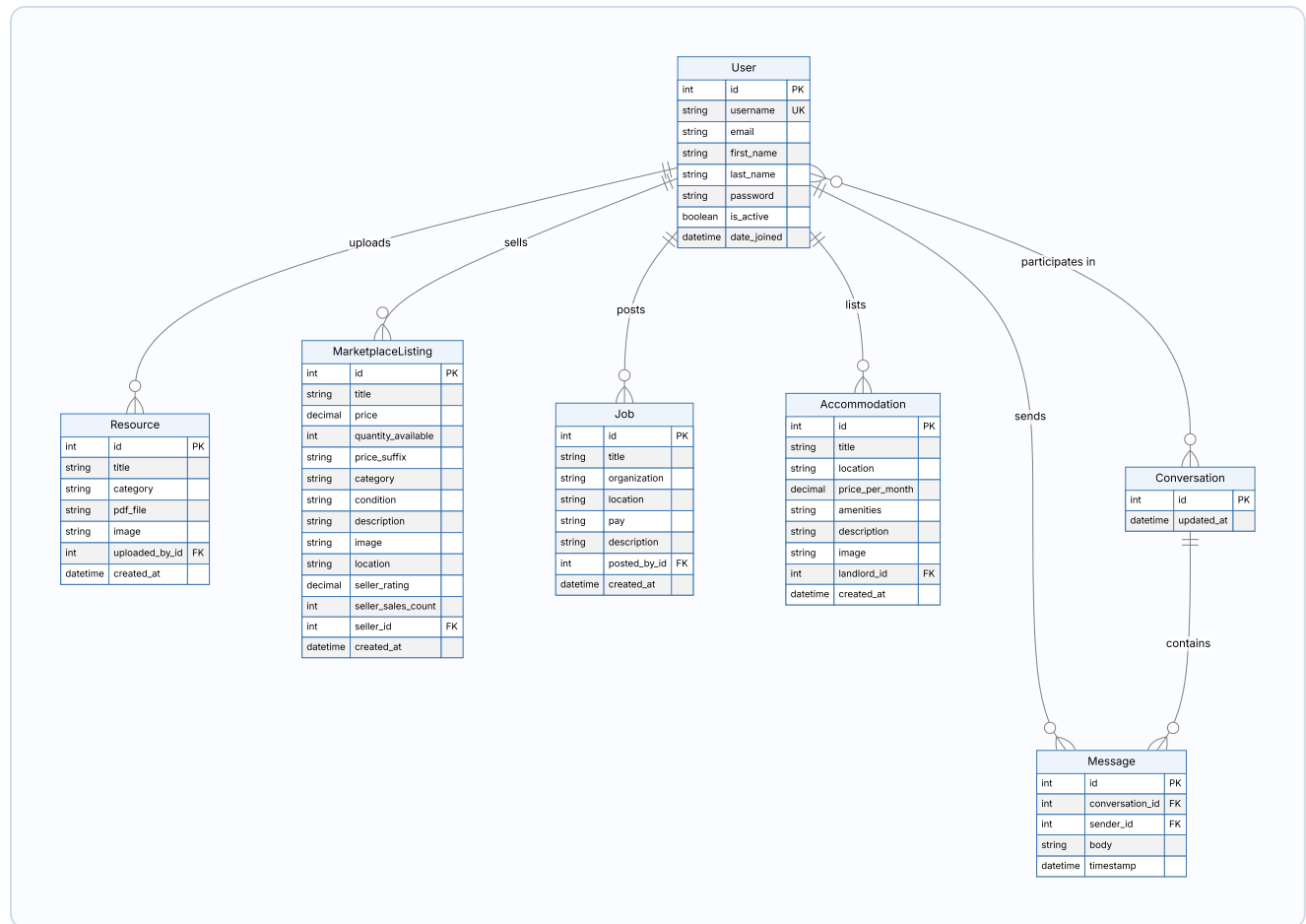
ID	Requirement	Specification
NFR-USAB-01	Responsive Breakpoints	System UI must adjust across three canonical viewport brackets: Mobile (<code>≤ 640px</code>), Tablet (<code>641px - 1024px</code>), Desktop (<code>≥ 1025px</code>).
NFR-USAB-02	Sidebar Navigation	Sidebar width set to 280px with a sliding drawer and dark backdrop on screens <code>< 1024px</code> , collapsible with zero screen displacement.
NFR-USAB-03	Long Name Handling	All user labels must support multi-line text wrapping (<code>overflow-wrap: break-word; word-break: break-word;</code>) and ellipsis fallback with tooltip <code>title</code> attributes.
NFR-USAB-04	Touch Targets	Interactive targets (buttons, pills, navigation links) must meet a minimum bounding box of 44 × 44 pixels on touch-enabled viewports.
NFR-USAB-05	Color Contrast	Color pairings must satisfy WCAG 2.1 Level AA contrast ratio of at least 4.5:1 for standard text and 3:1 for large headings and icons.

4.4 Reliability, Fault Tolerance, and Maintainability

ID	Requirement	Specification
NFR-REL-01	Media Fallback Grace	Missing or broken uploaded images must never produce broken image icons or unstyled layout collapses; cards must render branded SVG placeholders with category labels.
NFR-REL-02	Atomic Persistence	Database transactions must preserve data integrity; deletions must cascade appropriately (<code>models.CASCADE</code> on foreign keys).
NFR-MAINT-01	Automated Test Suite	The codebase must maintain 100% pass status across all automated unit and integration tests (<code>python manage.py test</code>), verifying all 28 core test cases.
NFR-MAINT-02	App Modularity	Distinct functionalities must remain separated across dedicated Django apps (<code>registration</code> , <code>dashboard</code> , <code>course_material</code> , <code>marketplace</code> , <code>housing</code> , <code>messaging</code>).

5. System Data Architecture & Entity-Relationship Model

5.1 Entity-Relationship Diagram



5.2 Data Dictionary

Entity: `Resource` (`course_material`)

Field	Type	Constraints	Description
<code>id</code>	AutoField	PK, Not Null	Unique identifier
<code>title</code>	CharField(200)	Not Null	Document or course title
<code>category</code>	CharField(50)	Choices: <code>TEXTBOOKS</code> , <code>NOTES</code> , <code>PAST_PAPERS</code> , <code>OTHER</code>	Material academic type
<code>pdf_file</code>	FileField	Path: <code>uploads/pdfs/</code> , Not Null	Physical PDF binary reference
<code>image</code>	ImageField	Path: <code>uploads/course_material/</code> , Null, Blank	Optional thumbnail cover image
<code>uploaded_by</code>	ForeignKey(User)	ON DELETE CASCADE	Contributor foreign key
<code>created_at</code>	DateTimeField	auto_now_add=True	Upload timestamp

Entity: MarketplaceListing (marketplace)

Field	Type	Constraints	Description
id	AutoField	PK, Not Null	Unique listing ID
title	CharField(200)	Not Null	Item or service headline
category	CharField(50)	Choices: textbooks , electronics , furniture , services	Market segment category
price	DecimalField(10,2)	Not Null, Min: 0.00	Monetary price in ZAR
price_suffix	CharField(20)	Null, Blank	Optional qualifier (e.g. /hr)
quantity_available	PositiveIntegerField	Default: 1	Current active inventory count
condition	CharField(100)	Default: Like new	Physical condition descriptor
description	TextField	Not Null	Detailed specification
image	ImageField	Path: uploads/marketplace/ , Null, Blank	Item photograph
seller	ForeignKey(User)	ON DELETE CASCADE	Seller user ID
seller_rating	DecimalField(3,2)	Default: 5.00	Seller community rating
seller_sales_count	IntegerField	Default: 0	Completed transactions
location	CharField(100)	Default: On campus	Meeting or pickup location
created_at	DateTimeField	auto_now_add=True	Listing creation timestamp

Entity: [Conversation](#) & [Message](#) ([messaging](#))

Entity	Field	Type	Constraints	Description
Conversation	id	AutoField	PK, Not Null	Unique thread ID
Conversation	participants	M2M(User)	Related: conversations	Conversation members
Conversation	hidden_for	M2M(User)	Related: hidden_conversations	Users who hid the thread
Conversation	updated_at	DateTimeField	auto_now=True	Timestamp of latest activity
Message	id	AutoField	PK, Not Null	Unique message ID
Message	conversation	ForeignKey(Conversation)	ON DELETE CASCADE	Parent conversation
Message	sender	ForeignKey(User)	ON DELETE CASCADE	Originating sender
Message	body	TextField	Not Null	Plaintext message payload
Message	hidden_for	M2M(User)	Related: hidden_messages	Users who deleted the message
Message	timestamp	DateTimeField	auto_now_add=True	Message dispatch timestamp

6. External Interface Requirements

6.1 User Interface (UI) System Architecture

- **Global Responsive Shell** ([base.html](#)):
- Modern fixed/sticky top navigation bar with quick brand link, mobile hamburger toggle, notifications, and profile action pill.
- Persistent 280px left sidebar navigation with user avatar, name resolution, handle, unread bell indicator, and module navigation links.

- Off-canvas overlay drawer on mobile viewports triggered by hamburger icon and closed by backdrop tap or close button.
- **Micro-Interactions & Feedback:**
- Dynamic JavaScript download toast on study material downloads.
- Synchronized CSS hover states with elevation shadows (`var(--shadow-card)` to `var(--shadow-soft)`).
- Native browser confirmation modals on destructive actions (`onsubmit="return confirm(...);"`).

6.2 Software Interfaces

- **Python / Django Framework:** Django 6.0 web engine handling routing, template compilation, CSRF middleware, and database ORM abstraction.
 - **Pillow (Python Imaging Library):** Image validation, EXIF normalization, and media processing.
 - **Relational Database Backend:** Interfaced through standard DB-API 2.0 driver.
 - **Web Server Gateway Interface (WSGI):** Compliance with PEP 3333 for standard web server execution.
-

7. Verification & Acceptance Criteria (Requirements Traceability)

Requirement ID	Requirement Summary	Verification Method	Verification Description / Test Scenario
FR-AUTH-01	User Registration	Unit & Integration Test	Submit valid and invalid registration forms. Verify DB persistence and auto-login in <code>test_auth.py</code> .
FR-AUTH-02	User Login	Unit & Integration Test	Post valid/invalid credentials; assert session cookie assignment and redirection to <code>/dashboard/</code> .
FR-AUTH-03	Logout Protection	Functional Test	Verify GET to logout fails or redirects; assert POST successfully terminates session and clears cookies.
FR-AUTH-04	Profile & Long Names	Manual UI Inspection	Create user with 35-character name; verify text wraps cleanly in 280px sidebar without overflow.
FR-ACAD-02	Resource Upload	Automated Test	Upload mock PDF via <code>ResourceUploadForm</code> ; verify record created in DB and stored under <code>uploads/pdfs/</code> .
FR-ACAD-03	PDF Download	Automated Test	Request <code>resource-download</code> URL; verify HTTP 200, Content-Type <code>application/pdf</code> , and attachment header.
FR-ACAD-04	Resource Deletion	Security Test	Attempt deletion as non-owner (expect 404/403). Delete as owner; verify record and physical file removed.
FR-MKT-02	Marketplace Listing	Automated Test	Submit listing with price and category; verify card renders on <code>/marketplace/marketplace/</code> .
FR-MKT-04	Stock Updating	Automated Test	Update stock from 2 to 0; assert card state switches from <code>In Stock</code> to <code>Currently sold out</code> .
FR-MSG-01	Conversation Start	Automated Test	Call <code>start_conversation</code> with another user; assert redirect to <code>chat-room</code> and Conversation entity created.
FR-MSG-04	Soft Delete	Integration Test	User A deletes conversation; assert thread excluded for User A but still visible and intact for User B.
NFR-SEC-02	IDOR Ownership Checks	Automated Test	Cross-user POST attempts on delete endpoints must reject non-owners.
NFR-REL-01	Image Fallbacks	Visual Regression Test	Load card with missing image path; assert SVG placeholder renders with category text on dark background.
NFR-MAINT-01	Test Suite Execution	Automated CLI Runner	Run <code>python manage.py test</code> ; assert all 28 automated test cases execute with zero failures (<code>OK</code>).

